AppVersal Golang Developer Intern - What to Learn

Overview

This is a Go-specific backend internship — a real stretch since Go isn't in your current stack at all. The JD explicitly emphasizes goroutines/channels/concurrency, so that's the single most important thing to be able to speak to. Your systems-level CS fundamentals (DSA, concurrency concepts from C/C++, REST API design from Node) transfer conceptually, but the language and its idioms are new.

MUST learn

Go language basics & goroutines/channels/concurrency (explicitly emphasized in the JD)

- What it is: Go is a statically-typed, compiled language built around simple syntax and first-class concurrency. Goroutines are lightweight threads (the runtime schedules thousands of them cheaply); channels are typed pipes goroutines use to communicate/synchronize instead of sharing memory directly — Go's core philosophy is "share memory by communicating, not communicate by sharing memory."
- Why the JD wants it: it's called out as an "emphasized requirement" separately from the general Golang skill — this is the single most important thing to be conversational about.
- The 20%: write one small program — spin up a few goroutines that each do work and send results back over a channel, read them all with a `select` or a simple loop. That one exercise covers goroutine creation (`go func()`), channel send/receive (`ch <- val`, `<-ch`), and buffered vs. unbuffered channels — the core vocabulary interviewers probe.
- Bridge: you already reason about async work (Node's event loop, Inngest's background jobs in Vibe) — goroutines are Go's answer to the same problem (do work concurrently), just with real OS-level concurrency instead of a single-threaded event loop, and channels replace Promises/callbacks as the coordination mechanism.

Go web/API basics: building REST APIs in Go

- What it is: Go's standard library (`net/http`) or a lightweight router (Gin, Echo, Chi) to define HTTP handlers, parse JSON request bodies, and return responses — structurally similar to Express.js but with explicit types and no middleware "magic."
- Why the JD wants it: "design and implement backend services using Go" + "build scalable APIs" is the core day-to-day responsibility.
- The 20%: build one tiny REST endpoint (a GET/POST pair) using `net/http` or Gin, returning JSON. This maps almost directly onto your Express.js route-handler mental model.
- Bridge: Express's `app.get('/route', (req, res) => ...)` maps to Go's `http.HandleFunc("/route", func(w, r) ...)` — same request/response shape, just typed and more explicit.

Microservices architecture

- What it is: splitting a system into independently deployable services (each owning its own data/logic), communicating over the network (REST/gRPC/message queues) rather than in-process function calls.
- Why the JD wants it: explicitly required, and Go is a very common microservices language (small binaries, fast startup, good concurrency for handling many requests).
- The 20%: you already have a genuine reference point — Echo's 7-workspace Turborepo monorepo is monorepo-organized but conceptually service-oriented (separate deployable pieces). Know the trade-off microservices make: independent scaling/deployment vs. added network latency and operational complexity vs. a monolith.

Apache Kafka

- What it is: a distributed event-streaming platform — producers publish messages to topics, consumers read them, enabling async, decoupled communication between services.
- Why the JD wants it: standard for inter-service communication in a microservices/Go backend stack.
- The 20%: know producer/consumer/topic/partition/consumer-group vocabulary and why it's used over a direct API call (decoupling, replay, buffering under load).
- Bridge: closest analog in your own work is Inngest (Vibe project) — durable, async background jobs. Kafka generalizes that same "decouple work from the request" idea to communication between separate services instead of one app's own background jobs.

GOOD to learn

WebSockets

- What it is: a persistent, bidirectional connection between client and server (unlike REST's request/response cycle) — used for real-time features (chat, live updates, notifications).
- Why the JD wants it: listed as a required skill, likely for real-time backend features.
- Bridge: Convex (used in Echo) gives you real-time reactivity already, conceptually similar to what WebSockets provide at a lower level — you understand the "server pushes updates to connected clients" pattern; WebSockets is just the raw protocol version of it.

Not worth deep prep

MongoDB (you already know PostgreSQL/SQL deeply; MongoDB's document model is a quick conceptual add, not core to this role); AWS (already genuine via Secrets Manager); Redis (already genuine from Echo/Vibe) — these three are already covered by your existing experience, just make sure they're visible on the resume (they are on this variant).